QueryAPIGate HOW-TO GUIDE 13 · ACCESS

Set up your first scoped API key

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=13-set-up-a-scoped-api-key

Time: 5 minutes. **You'll end up with:** a key limited to exactly one connection, read-only by default - instead of handing out the admin key (which can manage connections, saved queries and other keys, in addition to running any SQL) to every integration that just needs to run queries.

The footgun to avoid first

shell

```
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "oops"}'
```

Verified, and genuinely surprising the first time: a key created with no `connections` field at all gets access to **every** connection on the server, not none - it comes back as `"connections": "*"`. Confirmed directly: a key created with just `{"name": "oops"}` could immediately run SQL against a connection it was never told about. Omitting a field here means "unrestricted," not "no access" - always pass `connections` explicitly, even if it's `[]`.

The right way: explicit and scoped

shell

```
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "reporting-readonly", "connections": ["shop"]
}'
```

json

```
{"name": "reporting-readonly", "connections": ["shop"], "allow_writes": false, "active": true, ...,
 "secret": "sk_fIwXmahCwe1fEMciLfOnSul5tK6Egs3kes6X188Hdi4"}
```

Copy that secret now. Like every API key, it's generated by the server and shown exactly once - there is no way to retrieve it again later, only to revoke the key and create a new one. QueryAPIGate itself never stores it; only a SHA-256 hash is kept.

Verified, this one key, three ways:

```shell
# Works - its own connection, read-only
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: <secret>' -H 'Content-Type:
application/json' \
  -d '{"sql": "SELECT * FROM products", "connection_name": "shop"}'
# [{"id": 1, "name": "Widget"}]

# Refused - a different connection it was never granted
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: <secret>' -H 'Content-Type:
application/json' \
  -d '{"sql": "SELECT * FROM secrets", "connection_name": "secrets-db"}'
# {"error": "This API key is not permitted to use the connection 'secrets-db'"}

# Refused - allow_writes defaults to false
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: <secret>' -H 'Content-Type:
application/json' \
  -d '{"sql": "DELETE FROM products", "connection_name": "shop"}'
# {"error": "Only read-only statements (SELECT, WITH, SHOW, DESCRIBE, EXPLAIN) are allowed. ..."}
```

What a key actually looks like, in full

```shell
curl http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key'
```

```json
{
  "items": [{
    "name": "reporting-readonly",
    "connections": ["shop"], "collections": [], "queries": [], "allow_writes": false,
    "rate_limit": null, "allowed_ips": null, "allowed_write_ops": null, "allowed_tables": null,
    "created_from_role": null, "active": true, "expires_at": null, "expired": false,
    "created_at": "2026-10-01 06:21:12", "last_used_at": "2026-10-01 06:21:12",
    "example": false, "usage": {"queries": 1, "errors": 0, "rows": 2}
  }]
}
```

Never the secret itself - `GET /api/v1/api-keys` lists everything about a key except the one thing that would let someone use it. `last_used_at` is real and updates on actual use (confirmed above, right after calling it) - a way to tell a key that's actually alive from one nobody's touched in months, independent of its permissions.

Every field here is its own, deeper topic - this guide only covers the two most load-bearing ones (`connections`, `allow_writes`). See the next-steps list below for the rest.

Revoking it

```shell
curl -X DELETE http://127.0.0.1:5000/api/v1/api-keys/reporting-readonly -H 'X-API-Key: demo-key'
```

Verified: the key stops authenticating on its very next request - `{"error": "Unauthorized"}`, immediately, no grace period, no cache to wait out.

The admin key itself is just as strict about the boundary

```shell
curl http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: <a scoped key, not the admin one>'
# {"error": "This API key is not authorized to manage the server configuration", "code": "admin_only",
...}
```

Only administrators - the shared `QUERYAPIGATE_API_KEY`, or a named owner or admin (guide 43) - can create, list, update or delete keys at all - a scoped key, however permissive its own `connections` / `allow_writes` grants are, can never manage keys, connections, or saved queries. That boundary is absolute, not something any grant field widens.

Next steps

Each of these is the same `POST /api/v1/api-keys` call, just with one more field:

- Give a key access to a whole collection - `collections`.
- Give an external partner access to one query only - `queries`, with no `connections` grant at all.
- Create a role to stamp out several similarly-scoped keys - a reusable template for everything above.
- Restrict a key to specific tables - `allowed_tables`.
- Rate-limit a key - `rate_limit`.
- Restrict a key to specific source IPs - `allowed_ips`.
- Give a key an expiry date - `expires_at`.
- Allow a saved query to write data - `allow_writes`, `allowed_write_ops`, and the narrowest option of all: write access scoped to one named query.